

Topics in Quantitative Finance - Homework 4

Assigned: August 4, 2026. Due: Thursday, **August 6, 2026 by 1PM.**

Late homework **will not be accepted.**

Homework is to be done by each student individually. To receive full credit, you must email a completed copy of this Jupyter notebook to TAs at topics_in_qf@163.com by the due date and time. All codes must run correctly and solutions must be written up neatly in Markdown/LaTeX format. If you encounter problems with Jupyter notebook, please contact TA 李新宇 or 林文鑫.

Name: msj

Set up python environment

The following two cells load required python code into the notebook for the homework.

```
In [1]: # import required modules
import datetime
from datetime import datetime as dt
import numpy as np
from numpy import exp, log, sqrt
import scipy.stats as ss
from scipy.stats import norm
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.formula.api as sm
from option_analytics import OptionAnalytics
```

```
In [2]: # 本block功能是定义class OptionAnalytics:
# 助教提供版本在我的本地环境中报错，因此在此重新定义，
# 保留了本题所需的功能。如有测试需求可注释此block。

class OptionAnalytics:
    def __init__(self, option_chain, expiry, today):
        """
        option_chain: [calls_dataframe, puts_dataframe]
        expiry: 'YYYY-MM-DD' 字符串或 datetime 对象
        today: 'YYYY-MM-DD' 字符串或 datetime 对象
        """
        self.expiry = expiry
        self.expiry_dt = self._to_datetime(expiry, "expiry")
        self.today = self._to_datetime(today, "today")

        if len(option_chain) != 2:
```

```

        raise ValueError(
            "option_chain 必须包含两个 DataFrame: [calls, puts]"
        )

    self.calls = option_chain[0].copy()
    self.puts = option_chain[1].copy()

    required_columns = {"strike", "bid", "ask"}

    missing_calls = required_columns - set(self.calls.columns)
    missing_puts = required_columns - set(self.puts.columns)

    if missing_calls:
        raise KeyError(
            f"calls 数据缺少这些列: {sorted(missing_calls)}"
        )

    if missing_puts:
        raise KeyError(
            f"puts 数据缺少这些列: {sorted(missing_puts)}"
        )

    # Call strikes 和中间价
    self.ks_c = self.calls["strike"].to_numpy(dtype=float)
    self.cs = (
        self.calls["bid"].to_numpy(dtype=float)
        + self.calls["ask"].to_numpy(dtype=float)
    ) / 2

    # Put strikes 和中间价
    self.ks_p = self.puts["strike"].to_numpy(dtype=float)
    self.ps = (
        self.puts["bid"].to_numpy(dtype=float)
        + self.puts["ask"].to_numpy(dtype=float)
    ) / 2

    # 找出 call 和 put 共同交易的 strike, 并保证价格一一对应
    call_prices = self.calls[
        ["strike", "bid", "ask"]
    ].copy()

    put_prices = self.puts[
        ["strike", "bid", "ask"]
    ].copy()

    call_prices["call_mid"] = (
        call_prices["bid"] + call_prices["ask"]
    ) / 2

    put_prices["put_mid"] = (
        put_prices["bid"] + put_prices["ask"]
    ) / 2

    common = pd.merge(
        call_prices[["strike", "call_mid"]],
        put_prices[["strike", "put_mid"]],
        on="strike",
        how="inner"
    )

```

```

common = (
    common
    .dropna(subset=["strike", "call_mid", "put_mid"])
    .sort_values("strike")
    .drop_duplicates(subset="strike")
    .reset_index(drop=True)
)

if common.empty:
    raise ValueError(
        "call 和 put 数据中没有共同的 strike。"
    )

self.ks = common["strike"].to_numpy(dtype=float)
self.mids_call = common["call_mid"].to_numpy(dtype=float)
self.mids_put = common["put_mid"].to_numpy(dtype=float)

# 计算隐含波动率
tmp = self.imp_vols()

self.ivs = tmp["imp_vols"]
self.s_adj = tmp["s_adj"]
self.pv = tmp["pv"]

@staticmethod
def _to_datetime(value, variable_name):
    """把字符串或日期对象转换为 datetime。"""
    if isinstance(value, datetime.datetime):
        return value

    if isinstance(value, datetime.date):
        return datetime.datetime.combine(
            value,
            datetime.time()
        )

    if isinstance(value, str):
        try:
            return datetime.datetime.strptime(
                value,
                "%Y-%m-%d"
            )
        except ValueError as exc:
            raise ValueError(
                f"{variable_name} 必须使用 YYYY-MM-DD 格式。"
            ) from exc

    raise TypeError(
        f"{variable_name} 必须是 YYYY-MM-DD 字符串或日期对象。"
    )

# Put-call parity
def plot_parity(self):
    plt.figure(figsize=(9, 5))

    plt.plot(
        self.ks,
        self.mids_call - self.mids_put,
        "r.--"
    )

```

```

plt.ylabel(r"$C-P$", fontsize=12)
plt.xlabel(r"$K$", fontsize=12)
plt.title(f"Expiry: {self.expiry}", fontsize=15)

return None

# 检查期权价格关于 strike 的单调性和凸性
def plot_arb(self):
    fig, axes = plt.subplots(
        1,
        2,
        figsize=(12, 5),
        sharey=True
    )

    axes[0].plot(self.ks_c, self.cs, "bo--")
    axes[0].set_title("Call Options")
    axes[0].set_ylabel("Option mid price", fontsize=12)
    axes[0].set_xlabel("Strike", fontsize=12)

    axes[1].plot(self.ks_p, self.ps, "ro--")
    axes[1].set_title("Put Options")
    axes[1].set_xlabel("Strike", fontsize=12)

    return None

# 直接绘制数据中已有的 implied volatility
def plot_imp_vols1(self):
    if "impliedVolatility" not in self.calls.columns:
        raise KeyError(
            "calls 数据中没有 impliedVolatility 列。"
        )

    if "impliedVolatility" not in self.puts.columns:
        raise KeyError(
            "puts 数据中没有 impliedVolatility 列。"
        )

    ivs_c = self.calls["impliedVolatility"]
    ivs_p = self.puts["impliedVolatility"]

    plt.figure(figsize=(10, 6))

    plt.plot(
        self.ks_p,
        ivs_p,
        "ro--",
        label="Implied volatility from puts"
    )

    plt.plot(
        self.ks_c,
        ivs_c,
        "bo--",
        label="Implied volatility from calls"
    )

    plt.title(
        f"Expiration date: {self.expiry}",

```

```

        fontsize=15
    )

    plt.xlabel("Strikes", fontsize=12)
    plt.ylabel("Implied volatilities", fontsize=12)
    plt.legend()

    return None

# Black-Scholes call price
def bs_call(self, s, K, t, sigma, r=0):
    s = np.asarray(s, dtype=float)
    K = np.asarray(K, dtype=float)
    sigma = np.asarray(sigma, dtype=float)

    if t <= 0:
        raise ValueError("到期时间 t 必须大于 0。")

    d1 = (
        np.log(s / K)
        + (r + 0.5 * sigma**2) * t
    ) / (sigma * np.sqrt(t))

    d2 = d1 - sigma * np.sqrt(t)

    return (
        s * norm.cdf(d1)
        - K * np.exp(-r * t) * norm.cdf(d2)
    )

# 用二分法计算 call implied volatility
def bs_impvol_call(
    self,
    s0,
    K,
    T,
    C,
    r=0,
    tolerance=1e-10,
    max_iterations=300
):
    K = np.asarray(K, dtype=float)
    C = np.asarray(C, dtype=float)

    K, C = np.broadcast_arrays(K, C)

    if T <= 0:
        raise ValueError(
            "到期日必须晚于 today。"
        )

    if s0 <= 0:
        raise ValueError(
            "估计得到的 adjusted stock price 必须大于 0。"
        )

    if np.any(K <= 0):
        raise ValueError(
            "折现后的 strike 必须大于 0。"
        )

```

```

sigma_low = np.full(K.shape, 1e-10)
sigma_high = np.full(K.shape, 10.0)

for _ in range(max_iterations):
    sigma_mid = (
        sigma_low + sigma_high
    ) / 2

    price_mid = self.bs_call(
        s0,
        K,
        T,
        sigma_mid,
        r
    )

    price_is_too_low = price_mid < C

    sigma_low = np.where(
        price_is_too_low,
        sigma_mid,
        sigma_low
    )

    sigma_high = np.where(
        price_is_too_low,
        sigma_high,
        sigma_mid
    )

    if np.max(
        sigma_high - sigma_low
    ) < tolerance:
        break

    return (sigma_low + sigma_high) / 2

# 根据 put-call parity 计算 implied volatility
def imp_vols(self):
    df = pd.DataFrame({
        "CP": self.mids_call - self.mids_put,
        "Strike": self.ks
    })

    result = sm.ols(
        formula="CP ~ Strike",
        data=df
    ).fit()

    # 使用参数名称, 而不是 result.params[0]
    # 因此同时兼容旧版和新版 pandas
    s_adj = float(
        result.params.loc["Intercept"]
    )

    pv = -float(
        result.params.loc["Strike"]
    )

```

```

if s_adj <= 0:
    raise ValueError(
        f"回归得到的 s_adj={s_adj:.6f}, 必须大于 0。"
    )

if pv <= 0:
    raise ValueError(
        f"回归得到的 pv={pv:.6f}, 必须大于 0。"
    )

ks_pv = self.ks * pv

days_to_expiry = (
    self.expiry_dt - self.today
).days

if days_to_expiry <= 0:
    raise ValueError(
        "expiry 必须晚于 today。"
    )

imp_vols = self.bs_impvol_call(
    s0=s_adj,
    K=ks_pv,
    T=days_to_expiry / 365,
    C=self.mids_call,
    r=0
)

return {
    "imp_vols": imp_vols,
    "pv": pv,
    "s_adj": s_adj
}

# 绘制模型计算的 implied volatility
def plot_imp_vols2(self):
    ivs = np.asarray(self.ivs, dtype=float)

    mask = (
        np.isfinite(ivs)
        & (ivs > 0.001)
        & np.isfinite(self.ks)
        & (self.ks > 0)
    )

    if not np.any(mask):
        raise ValueError(
            "没有可用于绘图的有效隐含波动率。"
        )

    x = self.ks[mask]
    y = ivs[mask]

    plt.figure(figsize=(10, 6))

    log_moneyness = np.log(
        x / self.s_adj
    )

```

```

plt.plot(log_moneyness, y, "b.--")
plt.plot(log_moneyness, y, "r.")

plt.xlabel("Log moneyness", fontsize=12)
plt.ylabel("Implied volatilities", fontsize=12)
plt.title(
    "Implied Volatilities vs Log Moneyness",
    fontsize=15
)

return None

def __call__(self):
    return None

```

Historical volatility

```

In [3]: # Load daily OHLC data for Tesla from 2020-09-01 to 2024-07-26
        tsla = pd.read_csv('tsla_07292024.csv')
        tsla.index = tsla['Date']
        tsla = tsla.drop('Date', axis=1)

```

```

In [4]: # a quick look at the data
        tsla

```

Out[4]:

	Open	High	Low	Close	Adj Close	Volume
Date						
2020-09-01	167.380005	167.496674	156.836670	158.350006	158.350006	269523300
2020-09-02	159.663330	159.679993	135.039993	149.123337	149.123337	288528300
2020-09-03	135.743332	143.933334	134.000000	135.666672	135.666672	262788300
2020-09-04	134.270004	142.666672	124.006668	139.440002	139.440002	330965700
2020-09-08	118.666664	122.913330	109.959999	110.070000	110.070000	346397100
...	...	...	...	...	...	...
2024-07-22	244.210007	253.210007	243.750000	251.509995	251.509995	101225400
2024-07-23	253.600006	255.759995	245.630005	246.380005	246.380005	111928200
2024-07-24	225.419998	225.990005	214.710007	215.990005	215.990005	167942900
2024-07-25	216.800003	226.000000	216.229996	220.250000	220.250000	100636500
2024-07-26	221.190002	222.279999	215.330002	219.800003	219.800003	94483000

981 rows × 6 columns

1. (15pts)

Use the `python` code provided in the lecture, especially the part on historical volatilities, to

- a) plot daily close prices and daily log returns (using close prices) of Tesla in this period of time;
- b) calculate the following historical volatilities with moving window of 15 days: i) conditional standard deviation of log returns from close price, ii) Parkinson, iii) Garman-Klass, iv) Rogers-Satchell, by using the daily OHLC data stored in `tsla_07292024.csv`. Plot all the volatilities in one figure;
- c) illustrate the leverage effect by plotting the historical volatility series from b)i) and the price series in the same figure. Remember to properly rescale the price series.

Solution 1.

```
In [5]: # (1a) plot tsla close prices
plt.figure(figsize=(9, 6))
```

```

tsla['Close'].plot()
plt.title('TSLA Close Price (2020-09-01 to 2024-07-26)')
plt.ylabel('Close Price', fontsize=12)
plt.grid()
plt.show()

```

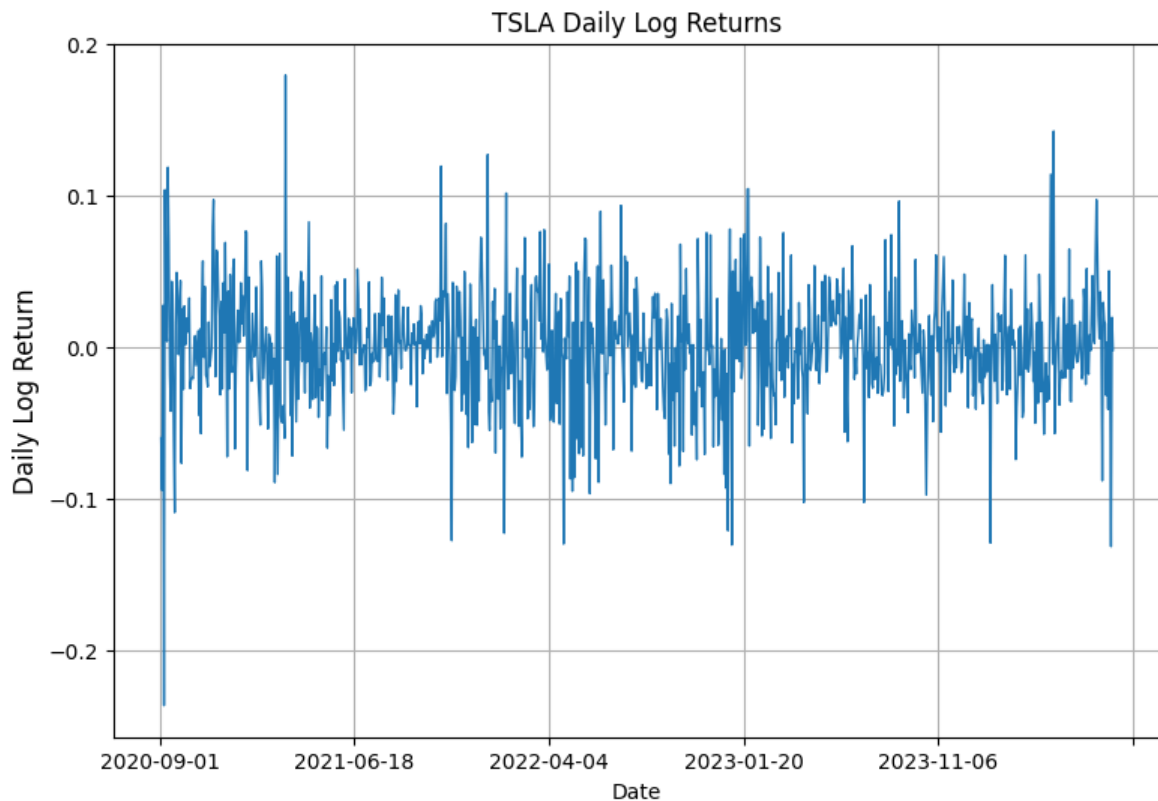


```

In [6]: # (1a) Log returns of tsla
r = log(tsla['Close']).diff()

plt.figure(figsize=(9, 6))
r.plot(lw=1)
plt.title('TSLA Daily Log Returns')
plt.ylabel('Daily Log Return', fontsize=12)
plt.grid()
plt.show()

```



```
In [7]: #(1b)
# Volatilities python class from Lecture
class Volatilities:
    def __init__(self, OHLC, n=10, N=252):
        self.n = n
        self.N = N
        self.OHLC = pd.DataFrame(OHLC)
        self.o = self.OHLC.Open
        self.h = self.OHLC.High
        self.l = self.OHLC.Low
        self.c = self.OHLC.Close
        self.r = log(self.OHLC['Close']).diff()
        self.vols_c = [np.nan for i in range(self.n)]
        self.vols_p = [np.nan for i in range(self.n)]
        self.vols_gk = [np.nan for i in range(self.n)]
        self.vols_rs = [np.nan for i in range(self.n)]

        for i in range(len(self.r) - self.n):
            self.vols_c += [self.r.iloc[i:(i+self.n)].std()*sqrt(self.N)]
            self.vols_p += [self.cal_vol_p(self.h.iloc[i:(i+self.n)], self.l.iloc[i:(i+self.n)])]
            self.vols_gk += [self.cal_vol_gk(self.o.iloc[i:(i+self.n)], self.h.iloc[i:(i+self.n)], self.l.iloc[i:(i+self.n)])]
            self.vols_rs += [self.cal_vol_rs(self.o.iloc[i:(i+self.n)], self.h.iloc[i:(i+self.n)], self.l.iloc[i:(i+self.n)])]
        self.vols = pd.DataFrame({'close': self.vols_c, 'parkinson': self.vols_p, 'garch': self.vols_gk, 'rs': self.vols_rs})
        self.vols.index = self.OHLC.index

    def cal_vol_p(self, H, L):
        return np.sqrt(((log(H) - log(L))**2).mean()/log(2)/4)

    def cal_vol_gk(self, O, H, L, C):
        term1 = ((log(O) - log(L))**2).mean()/2
        term2 = (2*log(2) - 1)*((log(C) - log(O))**2).mean()
        return np.sqrt(term1 + term2)

    def cal_vol_rs(self, O, H, L, C):
```

```

u, d, c = log(H) - log(O), log(L) - log(O), log(C) - log(O)
return np.sqrt((u*(u-c)).mean() + (d*(d-c)).mean())

```

```

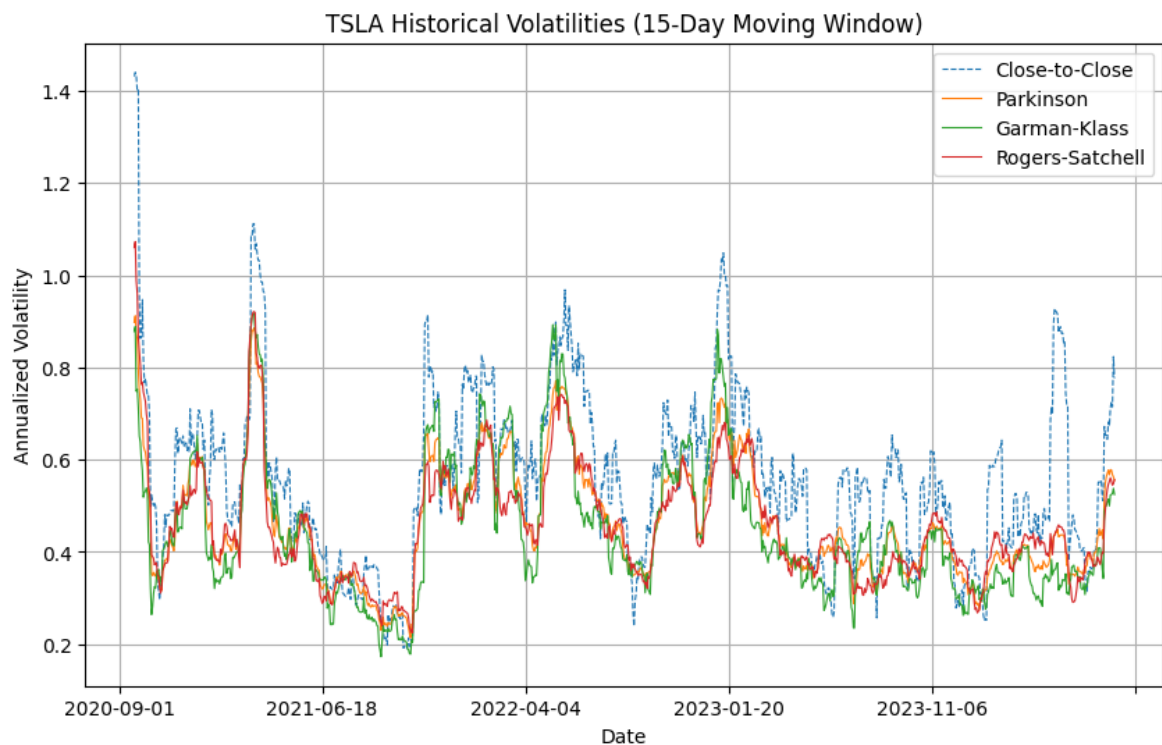
In [8]: # (1b) continued
# calculate volatilities using historical data

# use 15day window

tsla_vols = Volatilities(tsla, n=15, N=252)
vols = tsla_vols.vols

plt.figure(figsize=(10, 6))
vols['close'].plot(ls='--', label='Close-to-Close', lw=0.8)
vols['parkinson'].plot(lw=0.8, label='Parkinson')
vols['garman-klass'].plot(lw=0.8, label='Garman-Klass')
vols['rogers-satchell'].plot(lw=0.8, label='Rogers-Satchell')
plt.legend()
plt.title('Tesla Historical Volatilities (15-Day Moving Window)')
plt.xlabel('Date')
plt.ylabel('Annualized Volatility')
plt.grid()
plt.show()

```



```

In [9]: # (1c)
# leverage effect
# need to scale
tsla_close_scaled = (
    tsla['Close'] - tsla['Close'].mean()
) / (
    tsla['Close'].max() - tsla['Close'].min()
)

plt.figure(figsize=(9, 6))
tsla_close_scaled.plot(
    color='k',
    lw=1,

```

```

    label='Rescaled TSLA Close Price'
)
plt.plot(
    vols['close'],
    'b-.',
    lw=1,
    label='15-Day Close-to-Close Volatility'
)
plt.ylim([-0.6, 1])
plt.xlabel('Date')
plt.ylabel('Rescaled Price / Annualized Volatility')
plt.title('TSLA Price and Historical Volatility: Leverage Effect')
plt.grid()
plt.legend()
plt.show()

```

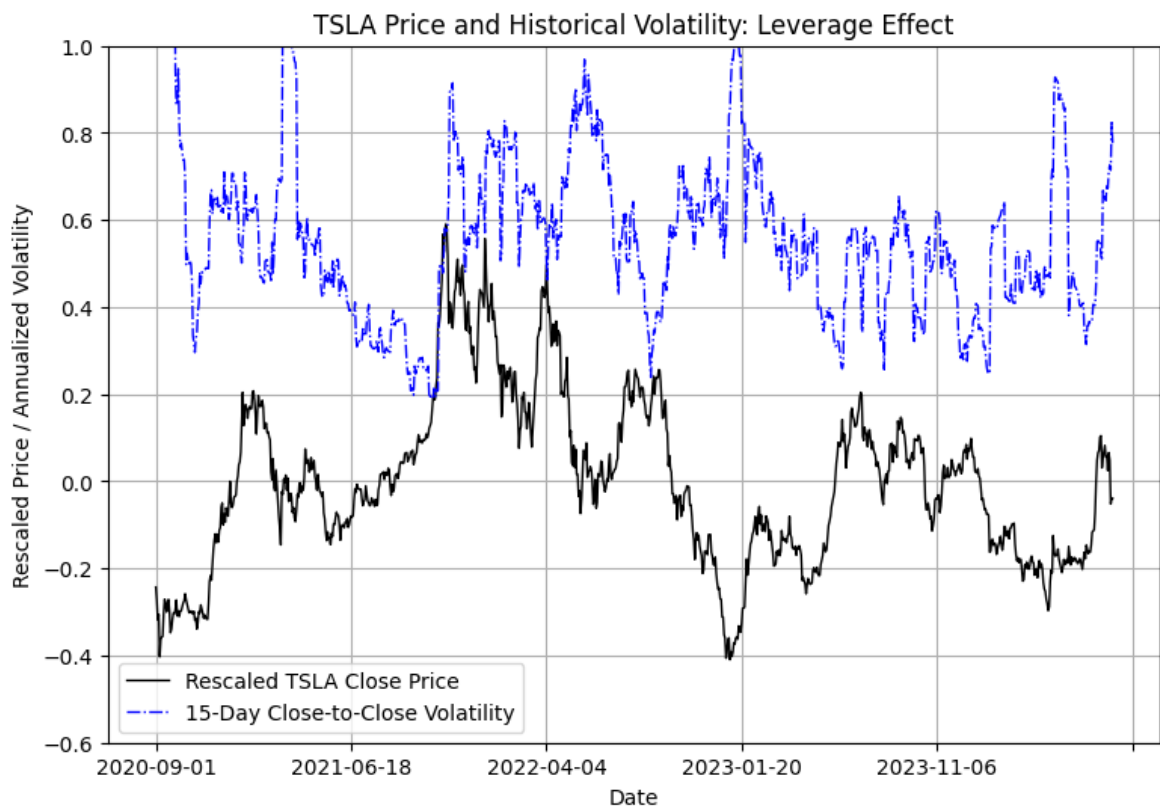


Illustration:

由上图，在Tesla股价下跌时，例如2022年下半年，波动率上升；当股价上升时，例如2021年，波动率下降。因此，图中股票价格与波动率呈现出一定的反向变化。这与课上所学的杠杆效应的特征基本一致。

这个现象的成因可以解释为：股价下跌时，公司权益价值下跌，财务杠杆被动上升，使得股票收益对市场信息的敏感度上升，从而使得波动率上升。

Implied volatility and VIX calculation

```
In [10]: # Load the saved data, previously downloaded from yahoo finance
aapl_calls = pd.read_csv('aapl_call_07252022.csv')
aapl_puts = pd.read_csv('aapl_put_07252022.csv')
today, expiry = '2022-07-25', '2022-08-26'
```

```
In [11]: # a Look at the first few rows of the data
aapl_calls.head()
```

Out[11]:

	contractSymbol	lastTradeDate	strike	lastPrice	bid	ask	change	p
0	AAPL220826C00070000	2022-07-21 15:28:15	70.0	83.60	83.05	83.20	0.0	
1	AAPL220826C00105000	2022-07-20 16:14:10	105.0	48.69	48.10	48.25	0.0	
2	AAPL220826C00110000	2022-07-20 17:09:38	110.0	42.77	43.20	43.35	0.0	
3	AAPL220826C00120000	2022-07-22 19:24:02	120.0	34.81	33.40	33.55	0.0	
4	AAPL220826C00125000	2022-07-22 18:50:21	125.0	29.50	28.45	28.75	0.0	



```
In [12]: # a Look at the last few rows of the data
aapl_puts.tail()
```

Out[12]:

	contractSymbol	lastTradeDate	strike	lastPrice	bid	ask	change	p
36	AAPL220826P00170000	2022-07-22 17:45:40	170.0	16.75	17.45	17.60	0.0	
37	AAPL220826P00180000	2022-07-19 19:40:09	180.0	29.35	26.95	27.15	0.0	
38	AAPL220826P00185000	2022-07-15 16:30:49	185.0	36.25	32.05	32.10	0.0	
39	AAPL220826P00190000	2022-07-21 18:24:42	190.0	34.75	36.90	37.10	0.0	
40	AAPL220826P00250000	2022-07-22 17:35:11	250.0	96.40	96.95	97.20	0.0	



2. (20pts)

Use the codes provided in lecture, especially the part on implied volatility, to

- a) determine if there exist arbitrage opportunities in this dataset, explain your answer;
- b) calculate the implied volatilities for AAPL using option data stored in `aapl_call_07252022.csv` and `aapl_put_07252022.csv`;

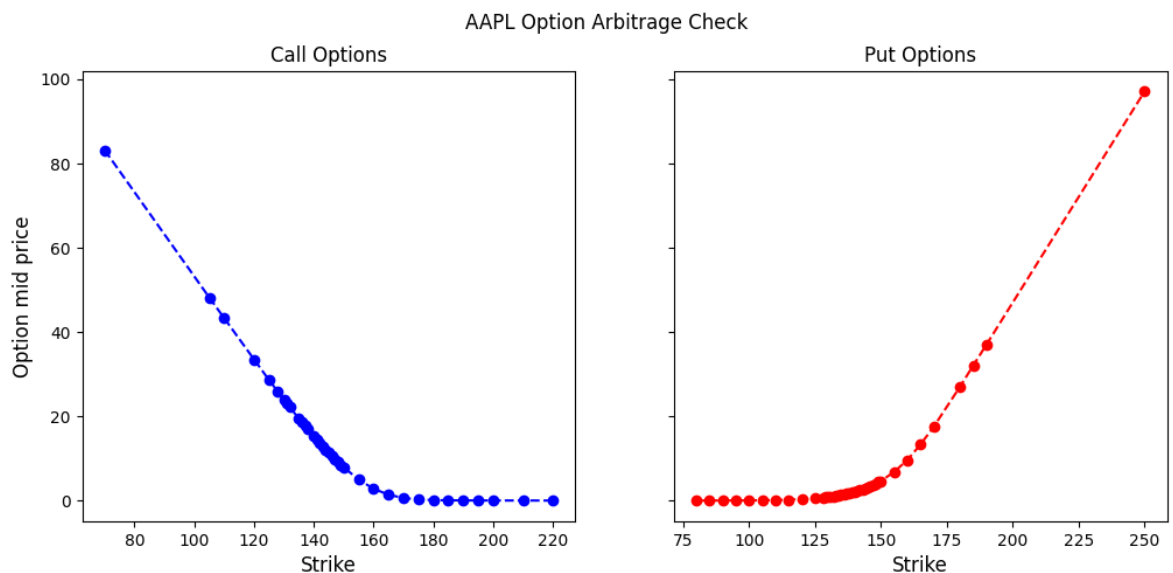
- c) plot the implied volatilities versus logmoneyness;
- d) use this dataset and the VIX formula to calculate the volatility index for AAPL.

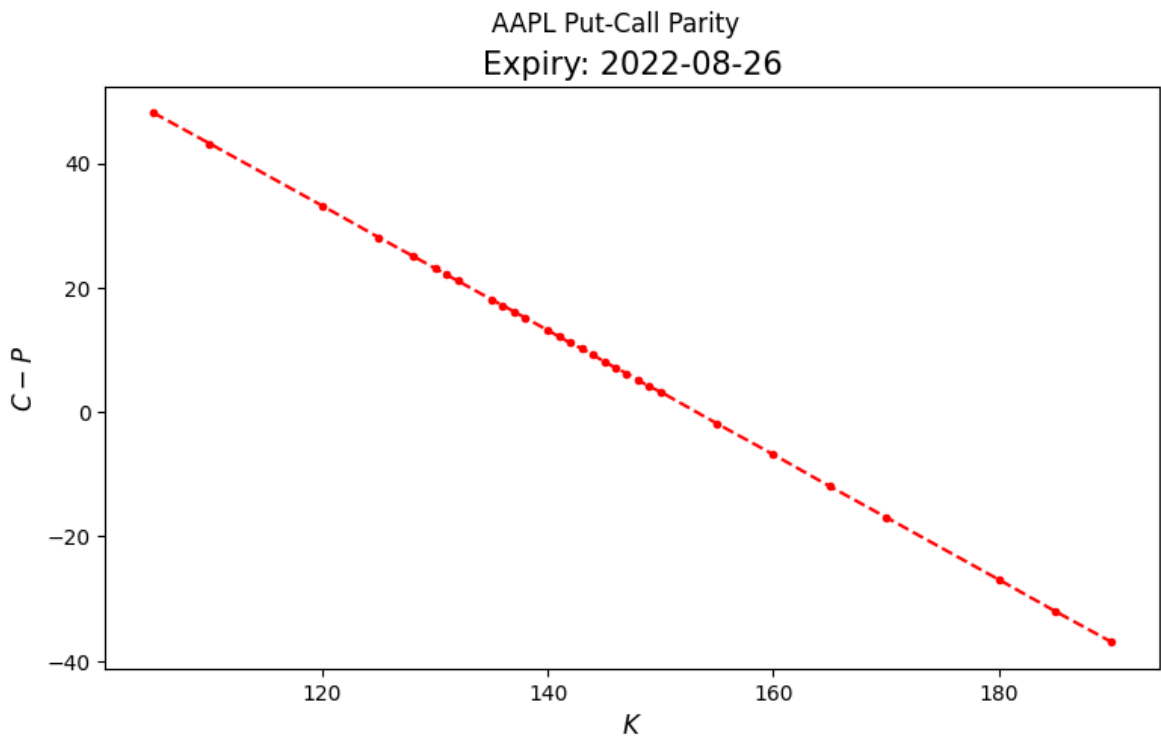
Solution 2.

```
In [13]: #(2a)
# Check for arbitrage opportunities
aapl_opt = OptionAnalytics([aapl_calls, aapl_puts], expiry, today)

# Plot option prices vs strike to check monotonicity and convexity
aapl_opt.plot_arb()
plt.suptitle('AAPL Option Arbitrage Check')
plt.show()

# Plot put-call parity
aapl_opt.plot_parity()
plt.suptitle('AAPL Put-Call Parity')
plt.show()
```





2(a) continue:

没有套利机会。

从上述结果我们可以看出：

1. 单调性： 如图：

- 看涨期权： 期权价格随行权价增加而呈现单调下降， 没有违反单调性要求。
- 看跌期权： 期权价格随行权价增加而单调上升， 没有违反单调性要求。

因此我们没有观察到可以通过买入低价卖出高价来套利的空间

2. 凸性 如图， 期权价格曲线均呈现凸性。 因此不能通过买入两端卖出中间的方式套利， 即没有蝶式套利机会。

3. Put-Call Parity 已知公式

$$C - P = S - K \times e^{-rT}$$

我们观察到图中点基本落在一条直线上， 没有出现明显偏离。 这说明不存在明显的平价套利机会。

综上所述， 该期权数据集中没有发现显著的套利机会。

```
In [14]:  #(2b)
# Compute implied volatilities
ivs = aapl_opt.ivs
ks = aapl_opt.ks
s_adj = aapl_opt.s_adj

print(f"Adjusted spot price from put-call parity: {s_adj:.2f}")
```

```

print(f"Number of implied volatilities: {len(ivs)}")

# Print the implied volatilities
print("\nImplied volatilities for AAPL options (Strike vs IV):")
for k, iv in zip(ks, ivs):
    print(f"Strike: {k:7.2f}    Implied Vol: {iv:.4f}")

```

Adjusted spot price from put-call parity: 153.19

Number of implied volatilities: 30

Implied volatilities for AAPL options (Strike vs IV):

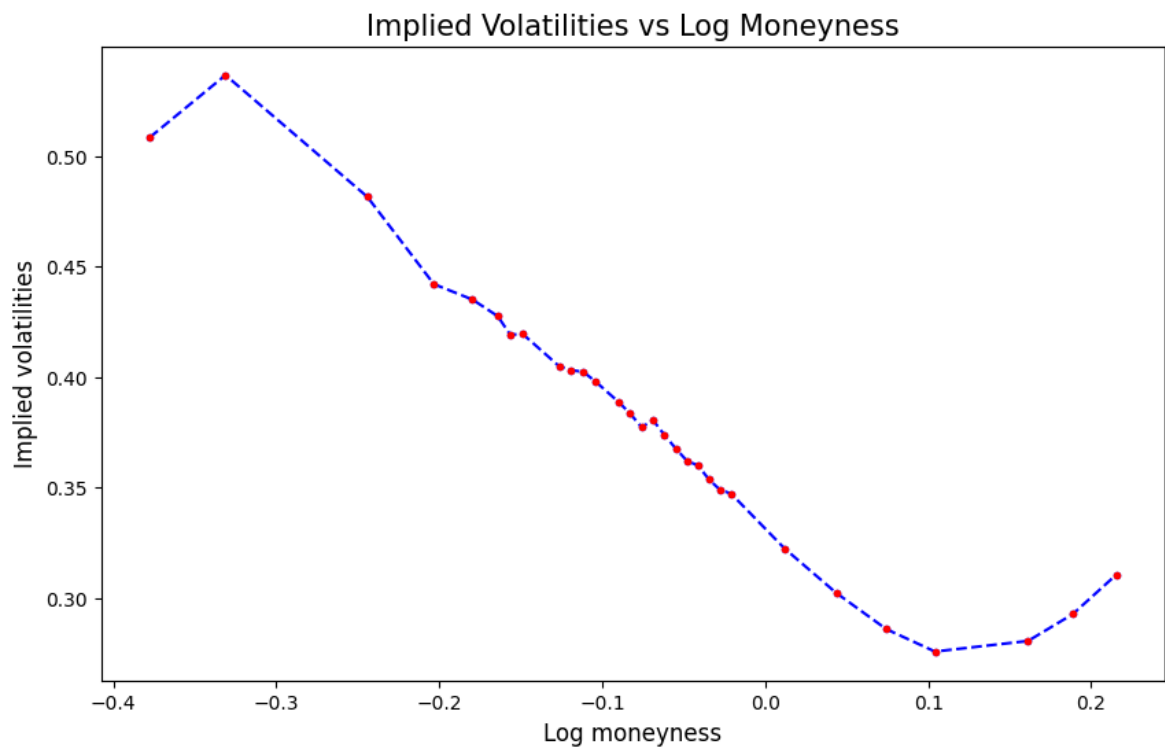
Strike: 105.00	Implied Vol: 0.5083
Strike: 110.00	Implied Vol: 0.5365
Strike: 120.00	Implied Vol: 0.4815
Strike: 125.00	Implied Vol: 0.4420
Strike: 128.00	Implied Vol: 0.4350
Strike: 130.00	Implied Vol: 0.4275
Strike: 131.00	Implied Vol: 0.4191
Strike: 132.00	Implied Vol: 0.4195
Strike: 135.00	Implied Vol: 0.4048
Strike: 136.00	Implied Vol: 0.4031
Strike: 137.00	Implied Vol: 0.4024
Strike: 138.00	Implied Vol: 0.3982
Strike: 140.00	Implied Vol: 0.3888
Strike: 141.00	Implied Vol: 0.3835
Strike: 142.00	Implied Vol: 0.3773
Strike: 143.00	Implied Vol: 0.3805
Strike: 144.00	Implied Vol: 0.3738
Strike: 145.00	Implied Vol: 0.3677
Strike: 146.00	Implied Vol: 0.3621
Strike: 147.00	Implied Vol: 0.3600
Strike: 148.00	Implied Vol: 0.3536
Strike: 149.00	Implied Vol: 0.3490
Strike: 150.00	Implied Vol: 0.3474
Strike: 155.00	Implied Vol: 0.3226
Strike: 160.00	Implied Vol: 0.3024
Strike: 165.00	Implied Vol: 0.2860
Strike: 170.00	Implied Vol: 0.2759
Strike: 180.00	Implied Vol: 0.2807
Strike: 185.00	Implied Vol: 0.2928
Strike: 190.00	Implied Vol: 0.3106

In [15]: *#(2c) plot implied vol vs logmoneyness*

```

aapl_opt.plot_imp_vols2()
plt.show()

```



```
In [16]: # (2d)
# recall the VIX formula
#  $VIX^2 = \frac{2}{T} \sum_i \frac{\Delta K_i}{K_i^2} Q_i(K_i) - \frac{1}{T} (F/K_0 - 1)^2$ ,
# where  $Q$  denotes prices of out-of-the-money options,  $F$  the forward price,
# and  $K_0$  the largest strike smaller than  $F$ 
```

```
In [17]: # (2d)
# Calculate a VIX-style volatility index for AAPL

# Time to expiration
today_dt = dt.strptime(today, '%Y-%m-%d')
expiry_dt = dt.strptime(expiry, '%Y-%m-%d')

days_to_expiry = (expiry_dt - today_dt).days
T = days_to_expiry / 365.0

# Get data from aapl_opt
pv = float(aapl_opt.pv)
s_adj = float(aapl_opt.s_adj)

# Forward price from put-call parity
F = s_adj / pv

strikes = np.asarray(aapl_opt.ks, dtype=float)
calls = np.asarray(aapl_opt.mids_call, dtype=float)
puts = np.asarray(aapl_opt.mids_put, dtype=float)

# Sort all arrays by strike
idx = np.argsort(strikes)

strikes_sorted = strikes[idx]
calls_sorted = calls[idx]
puts_sorted = puts[idx]
```

```

# K0: Largest strike strictly smaller than F,
# following the formula stated in the homework
K0_candidates = strikes_sorted[strikes_sorted < F]

if len(K0_candidates) == 0:
    raise ValueError("There is no strike below the forward price F.")

K0 = np.max(K0_candidates)

# Compute Delta K_i
n = len(strikes_sorted)

if n < 3:
    raise ValueError("At least three strikes are required.")

delta_K = np.empty(n, dtype=float)

delta_K[0] = (
    strikes_sorted[1] - strikes_sorted[0]
)

delta_K[-1] = (
    strikes_sorted[-1] - strikes_sorted[-2]
)

delta_K[1:-1] = (
    strikes_sorted[2:] - strikes_sorted[:-2]
) / 2.0

# Select out-of-the-money option prices Q(K)
Q = np.empty(n, dtype=float)

for i, k in enumerate(strikes_sorted):

    if k < K0:
        # OTM puts
        Q[i] = puts_sorted[i]

    elif k > K0:
        # OTM calls
        Q[i] = calls_sorted[i]

    else:
        # Cboe convention: at K0, use the average of the call and put midpoint p
        Q[i] = (
            calls_sorted[i] + puts_sorted[i]
        ) / 2.0

# Compute the summation term
contributions = (
    delta_K
    / strikes_sorted**2
    * Q
)

sum_term = np.sum(contributions)

```

```

# Apply the formula given in the homework
vix_squared = (
    (2.0 / T) * sum_term
    - (1.0 / T) * (F / K0 - 1.0)**2
)

# Only tolerate tiny negative values caused by floating-point error
if vix_squared < -1e-10:
    raise ValueError(
        f"Calculated variance is negative: {vix_squared:.8f}"
    )

vix_squared = max(vix_squared, 0.0)

vix_decimal = np.sqrt(vix_squared)
vix_percent = 100 * vix_decimal

print(f"Days to expiry: {days_to_expiry}")
print(f"Time to expiry T: {T:.6f}")
print(f"Discount factor: {pv:.6f}")
print(f"Forward price F: {F:.4f}")
print(f"K0: {K0:.2f}")
print(f"Number of strikes used: {n}")
print(f"AAPL volatility index: {vix_decimal:.4f}")
print(f"AAPL volatility index in percent: {vix_percent:.2f}%")

```

```

Days to expiry: 32
Time to expiry T: 0.087671
Discount factor: 1.000507
Forward price F: 153.1132
K0: 150.00
Number of strikes used: 30
AAPL volatility index: 0.3528
AAPL volatility index in percent: 35.28%

```